CS-2001
Data Structures
Fall 2023

Graph Algorithm (MST, SSSP)

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Minimum Spanning Tree

- A **tree** is a **connected graph** without cycles
- A **connected graph is a tree** if and only if it has N vertices and N-1 edges
- A subgraph that spans (reaches out to) all vertices of a graph is called a **spanning subgrap**h
- A subgraph that is a tree and that spans (reaches out to) all vertices of the original graph is called a **spanning tree**
- Among all the spanning trees of a weighted and connected graph, the one (possibly more) with the least total weight is called a **minimum spanning tree (MST)**

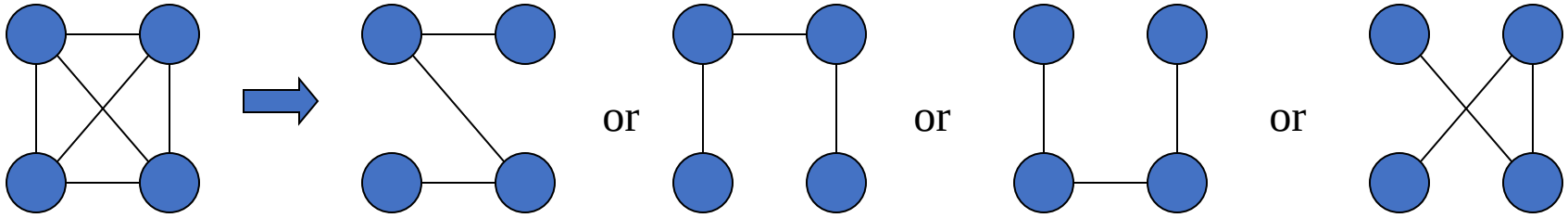
Spanning Trees

A spanning tree of a graph is just a subgraph that contains all the vertices and is a tree.

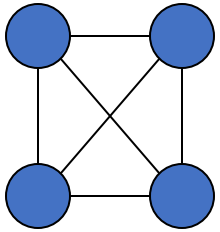
A graph may have many spanning trees.

Graph A

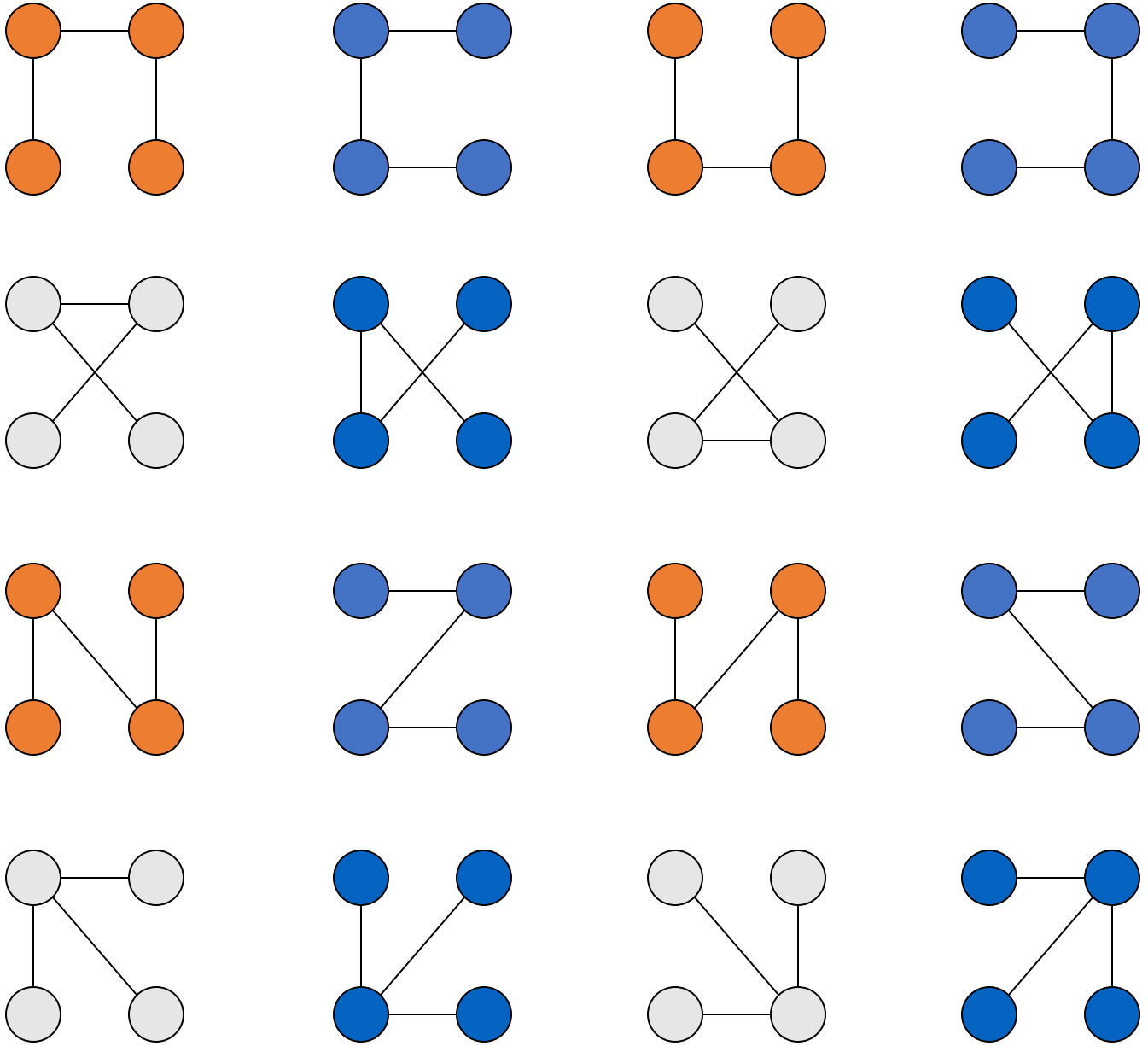
Some Spanning Trees from Graph A



Complete Graph



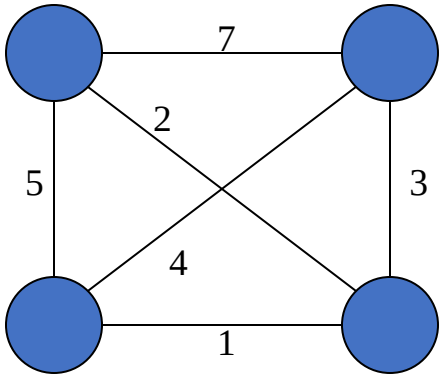
All 16 of its Spanning Trees



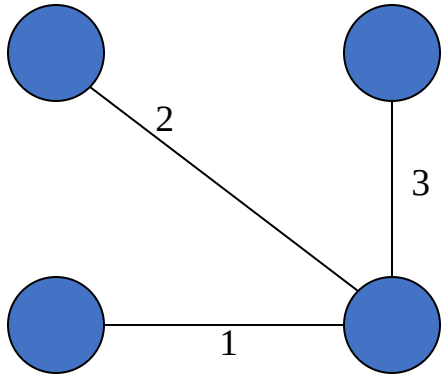
Minimum Spanning Trees

The Minimum Spanning Tree for a given graph is the Spanning Tree of minimum cost for that graph.

Complete Graph



Minimum Spanning Tree



Minimum Spanning Tree

- Is MST Unique???
  - Yes if edge lengths are distinct
  - No Otherwise

- In MST there is a single path between any two vertices??? Is this true.
  - Yes because there is no cycle

Algorithms for Obtaining the Minimum Spanning Tree

- Prim's Algorithm
- Kruskal's Algorithm

Prim's Algorithm

- This algorithm starts with one node
- It then, one by one, adds a node that is unconnected to the new graph
- Each time selecting the node whose connecting edge has the smallest weight out of the available nodes' connecting edges

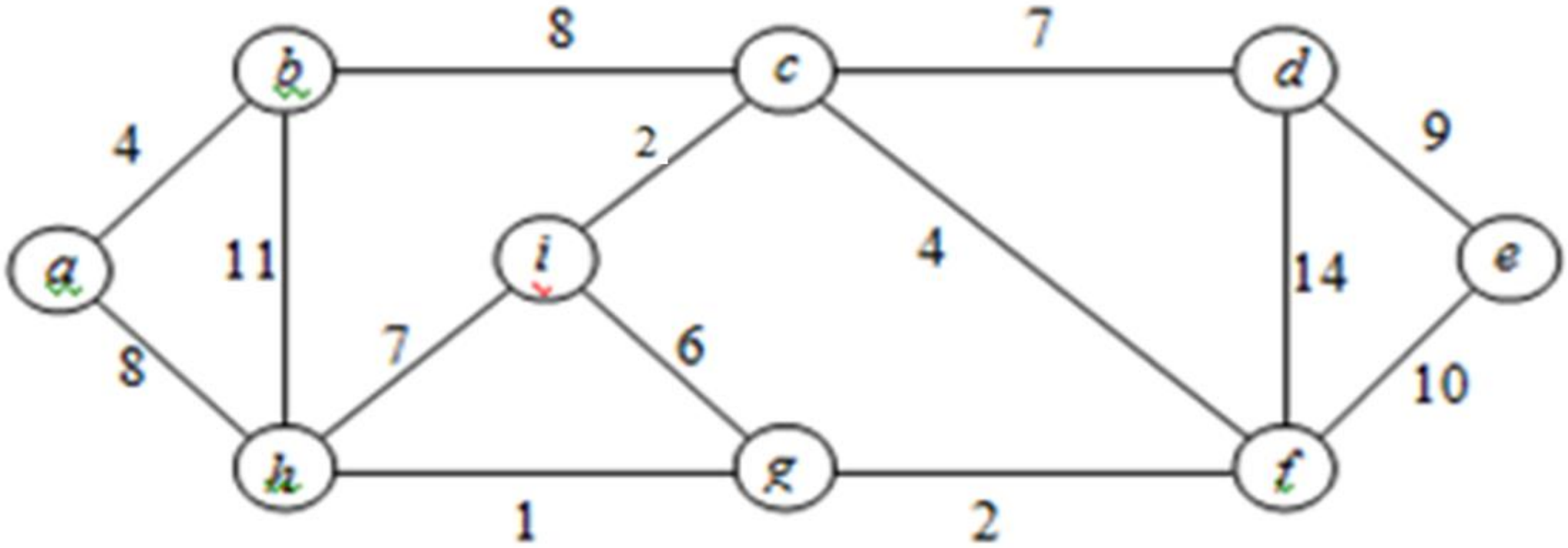
Steps:

- We now start the iterations.
- We select the node that has the least distance, say ‘u’.
  - In the first loop, the source is selected because its distance is zero
  - In subsequent iterations this process ensures that the minimum weight edge is selected. We now include the node in the tree.
- Then we relax all the nodes connected to the node
  - This means that we see for each node v connected to u whether the distance of v is greater than the weight of edge (u,v)
  - If it is, we assign w(u,v) to the distance of v
  - This step ensures that after each new node is added to the tree, the distance of the remaining nodes from the tree is updated
- Thus when we have all the edges in the tree, our minimum spanning tree is ready

PRIM'S Algorithm

```
1.  PRIM(G, w, r)
2.  for each u belongs to V[G]
    3.  do key[u] ← ∞
    4.  π[u] ← NIL  (Predecessor)
5.  key[r] ← 0
6.  Q ← V[G] // build min heap
7.  while Q ≠ ∅
    8.  do u ← EXTRACT-MIN(Q)
    9.  for each v belongs to Adj[u]
        10.do if v belongs to Q and w(u, v) < key[v]
            11. then π[v] ← u
            12. key[v] ← w(u, v)
// modify node and maintain heap property
```

Prim's Algorithm Example



Execution of Prim's Algorithm

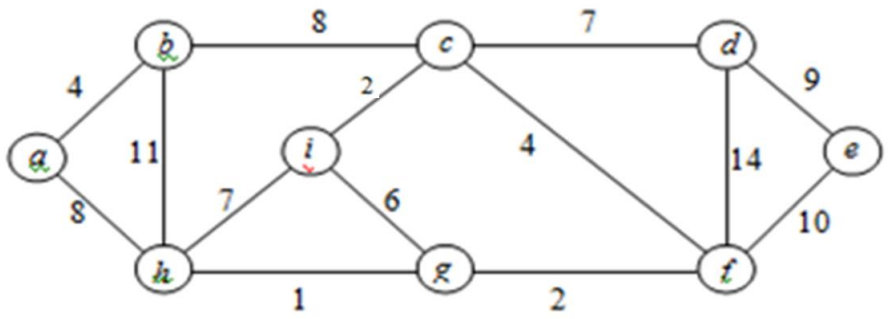
For Loop

V{G} = {a,b,c,d,e,f,g,h,i}

key[a] = ∞, key [b] = ∞, key [c] = ∞
key[d] = ∞, key [e] = ∞, key[f] = ∞
key[g] = ∞, key[h] = ∞, key [i] = ∞

As no vertex has been connected to tree

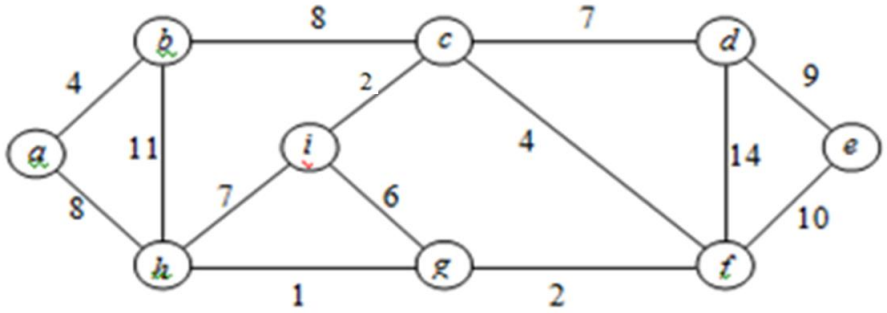
parent of [a] = Nil, parent of [b] = Nil,
parent of [c] = Nil, parent of [d] = N
parent of [e] = Nil, parent of [f] = N
parent of [g] = Nil, parent of [h] = N
parent of [i] = Nil,
key [a] = 0
Vertices in G = Q = {a,b,c,d,e,f,g,h,i}

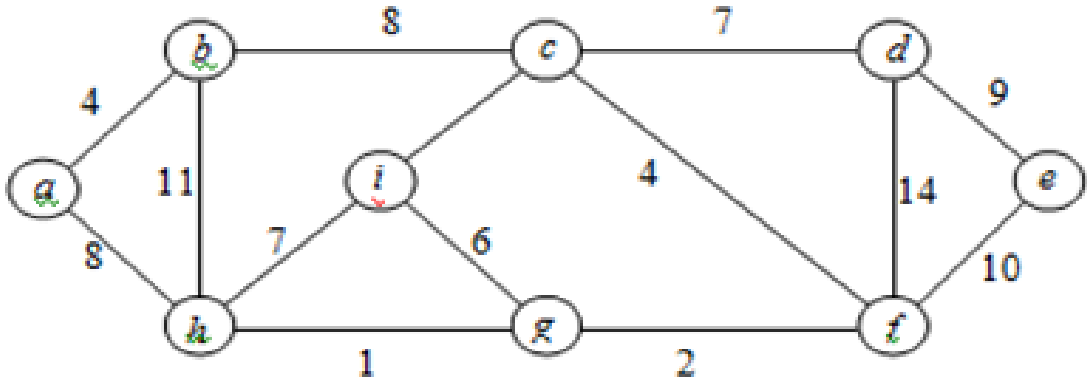


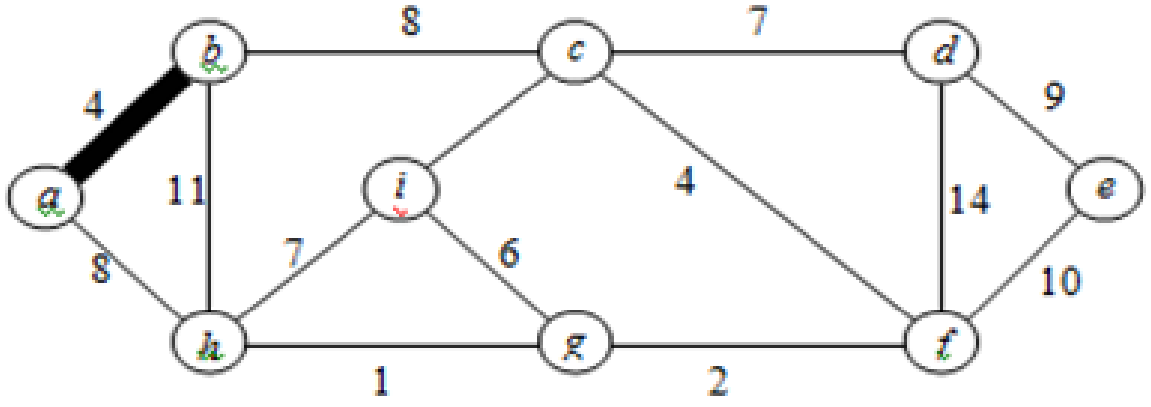
while Q != φ		*Extract u*	Adj {u} = v in Q	For each V ϵ Adj *u*		
				if	∏ {v}	key {v}
1	T	a	b, h	T, T	a	key[b] = 4, key[h] = 8
2	T	b	c, h	T, F	b	key[c] = 8
3	T	c	d, f, i	T, T, T	c	key[d] = 7, key[f] = 4, key[i] = 2
4	T	i	g, h	T, T	i	key[g] = 6, key[h] = 7
5	T	f	d, g, e	F, T, T	f	key[e] = 10, key[g] = 2
6	T	h	T	T	g	key[h] = 1
7	T	h	-		-	-
8	T	d	e	T	d	key[e] = 9
9	T	e	-	-	-	-

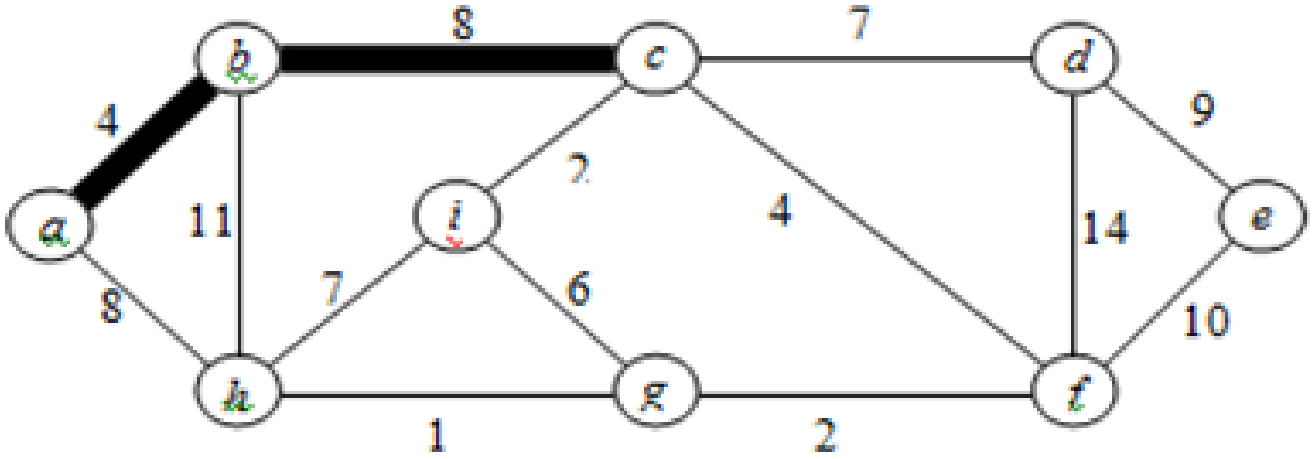
Removing u from set Q

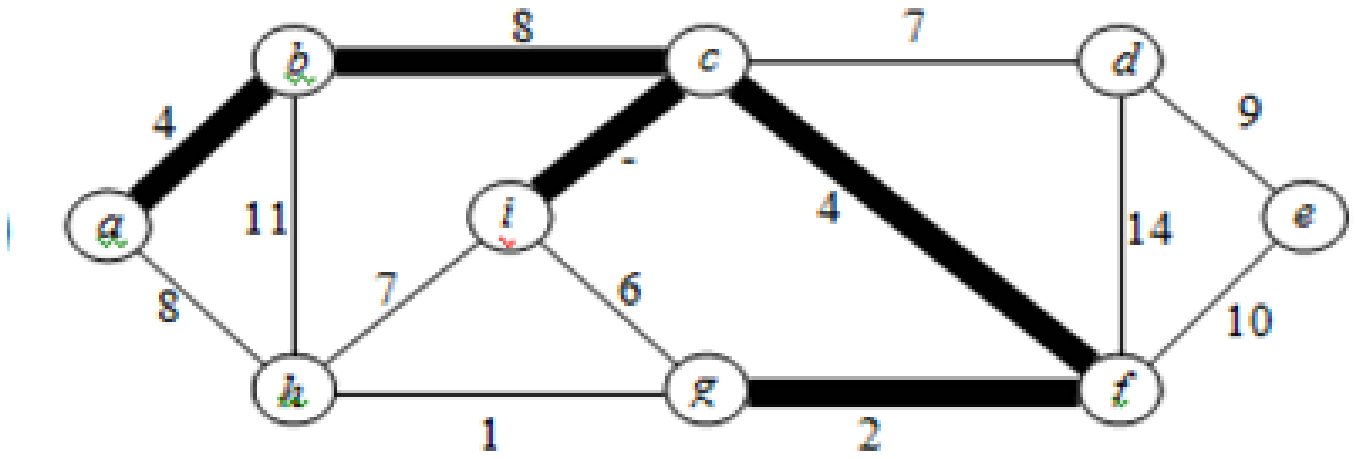
- **Add it to the set = vertices in the MST= {a,b,c,i,f,g,h,d,e}**
- **Thus adding (∏{u}, u) edge to A**

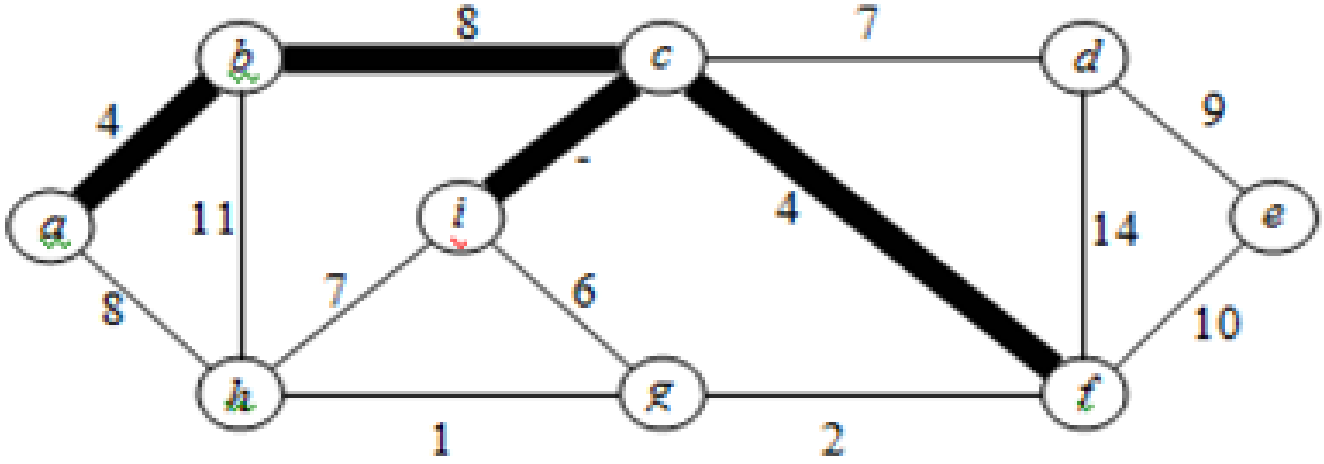


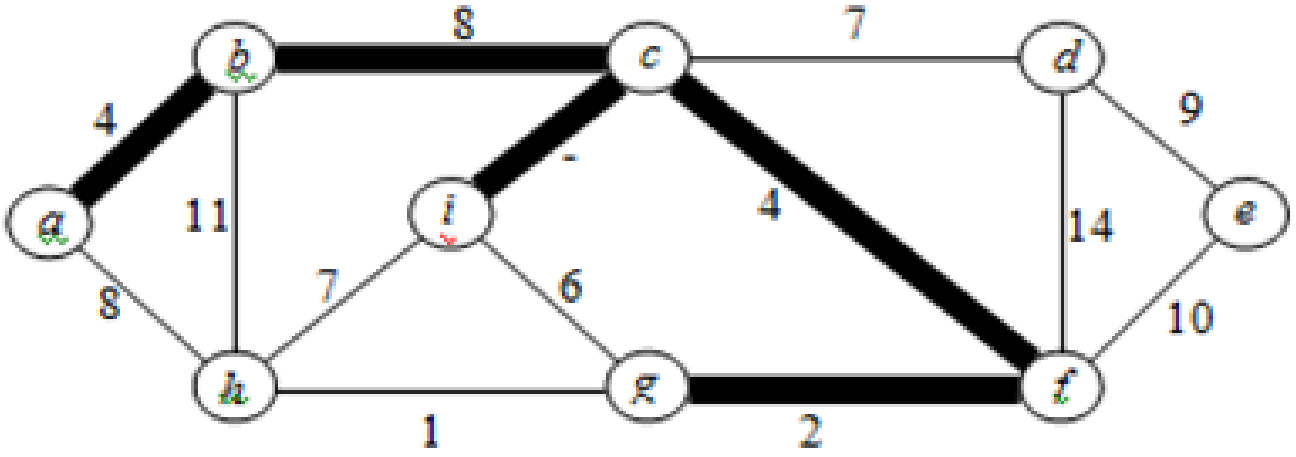

a
b
c
d
e
f
g
h
i
4
8
8
11
7
1
6
4
2
14
9
10

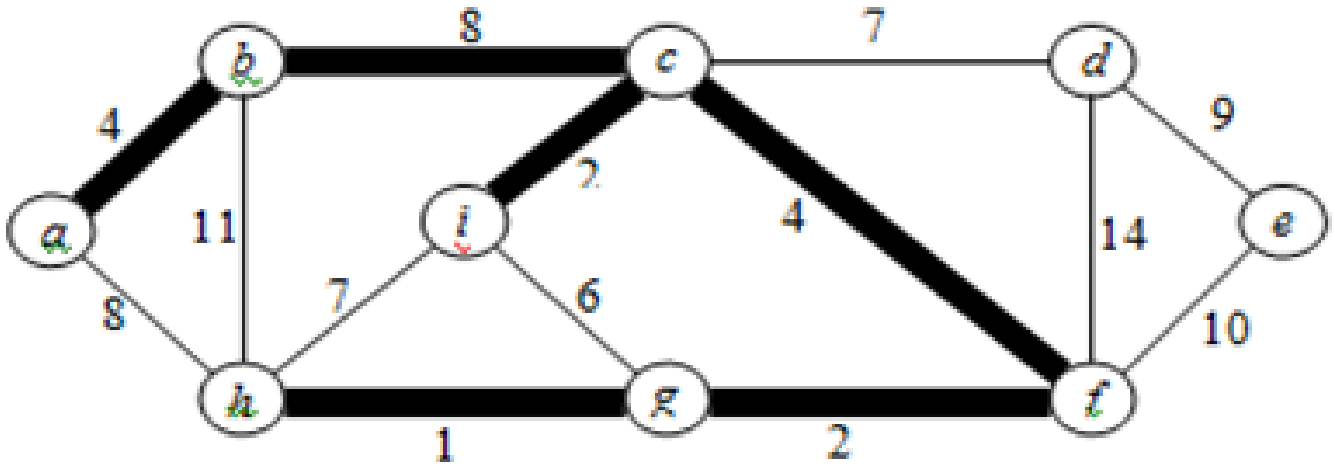

a
b
c
d
e
f
g
h
i
4
8
8
11
7
1
6
4
2
14
9
10

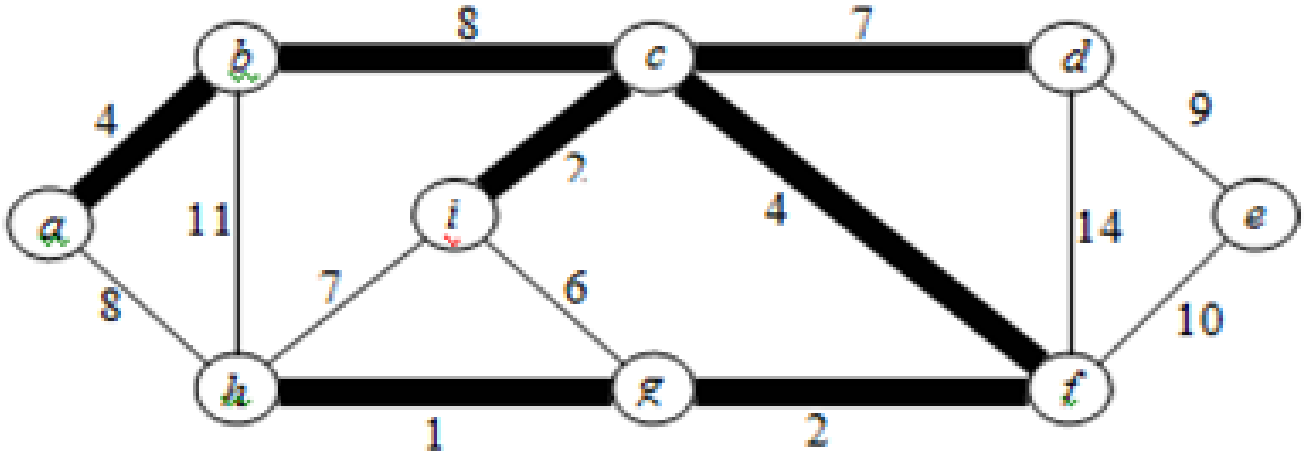

8
7
b
c
d
4
9
2
a
11
i
4
14
e
8
7
6
10
h
g
f
1
2

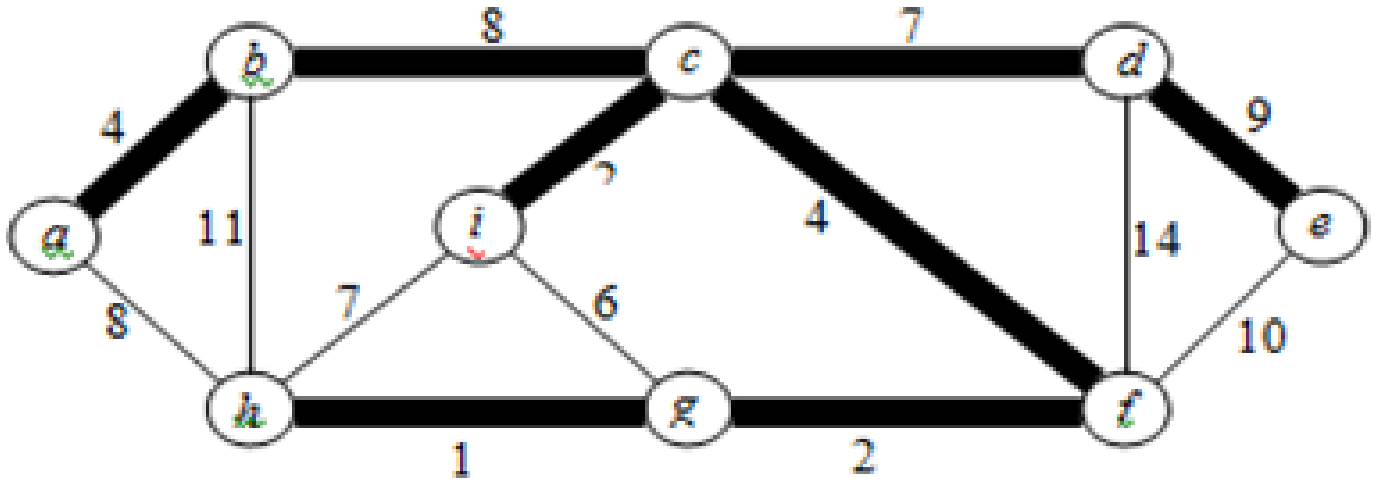

8
7
b
c
d
4
9
a
11
i
4
14
e
8
7
6
10
h
g
f
1
2


a
b
c
d
e
f
g
h
i
4
8
8
11
7
-
4
6
7
14
9
10
1
2


a
b
c
d
e
f
g
h
i
4
8
8
11
7
-
4
6
7
14
9
10
1
2


a
b
c
d
e
f
g
h
i
4
8
11
8
7
2
4
14
9
7
6
10
1
2

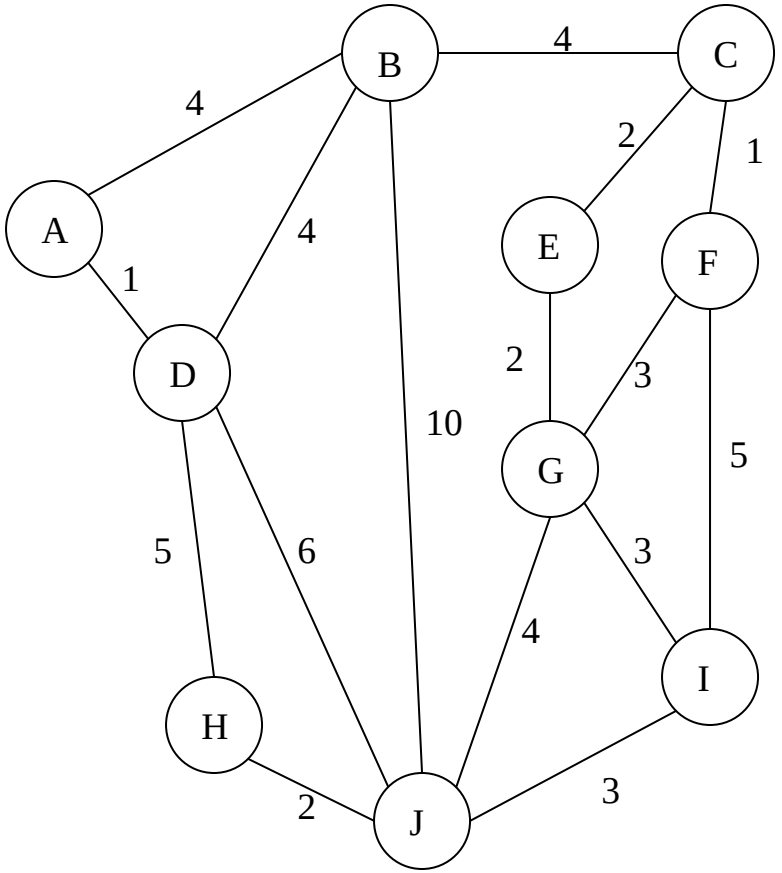

a
b
c
d
e
f
g
h
i
4
8
11
8
7
2
4
14
9
7
6
10
1
2



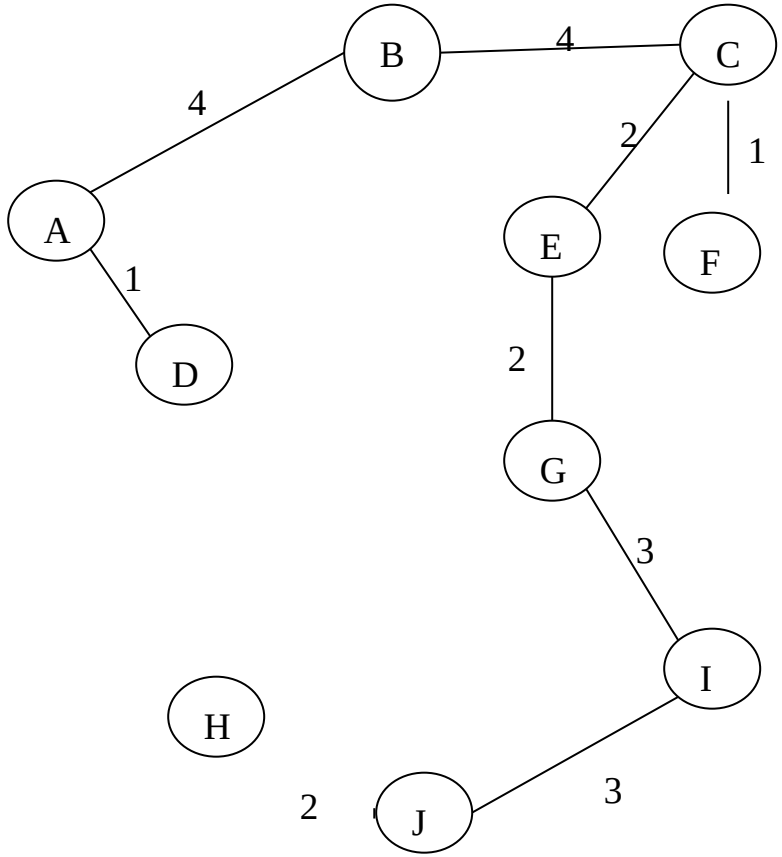
Safe Edges

Task

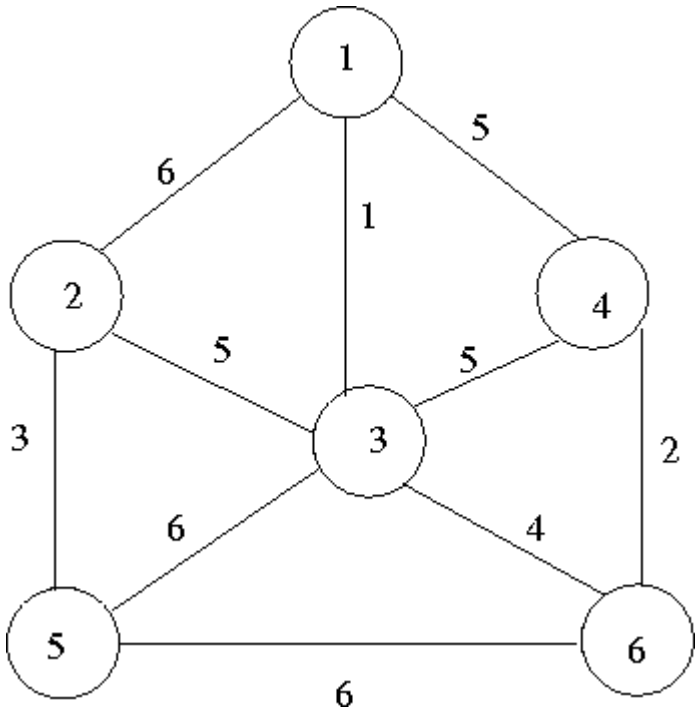
Complete Graph



MST



Home Task



Shortest Path Problem

Dijkstra's Algorithm

Shortest Path Problem

- In graph theory, the shortest path problem is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.

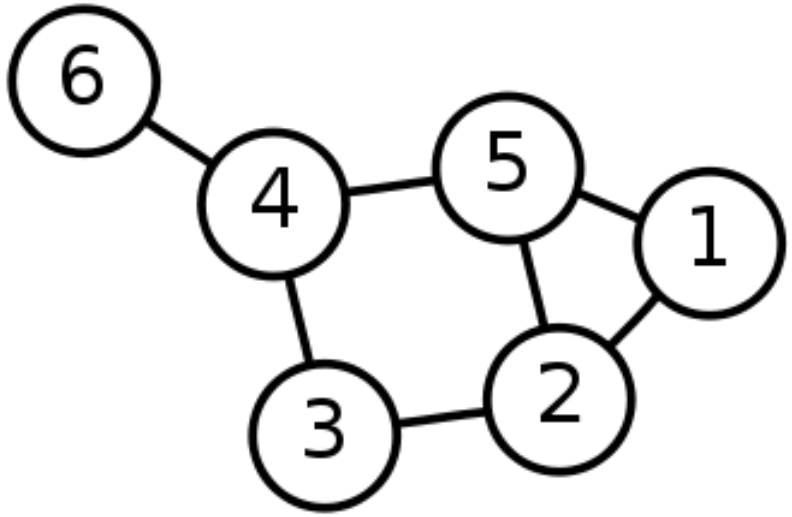
Shortest Path Problem

- Single-source shortest path problem
- Given a node A, find the shortest path from A to every other node in the graph

- All-pair shortest path problem
- Find the shortest path between every pair of nodes in the graph

Single-Source Shortest Path Problem

- Single-Source Shortest Path Problem - The problem of finding shortest paths from a source vertex v to all other vertices in the graph.



Dijkstra's Algorithm

- Similar to (BFS) Breadth-First Search
  - Grow a tree gradually, advancing from vertices taken from a queue
- Also similar to Prim's algorithm for MST
  - Use a priority queue keyed on d[v]

Dijkstra's algorithm

- **Dijkstra's algorithm** - is a solution to the single-source shortest path problem in graph theory
- Works on both **directed** and **undirected** graphs. However, all edges must have nonnegative weights
- **Approach:** Greedy
- **Input:** Weighted graph G={E,V} and source vertex v ∈ V, such that all edge weights are nonnegative
- **Output:** Lengths of shortest paths (or the shortest paths themselves) from a given source vertex v ∈ V to all other vertices

Dijkstra's Algorithm

- **The idea is to visit the nodes in order of their closeness to *A***; visit A first, then visit the closest node to A, then the next closest node to A, and so on

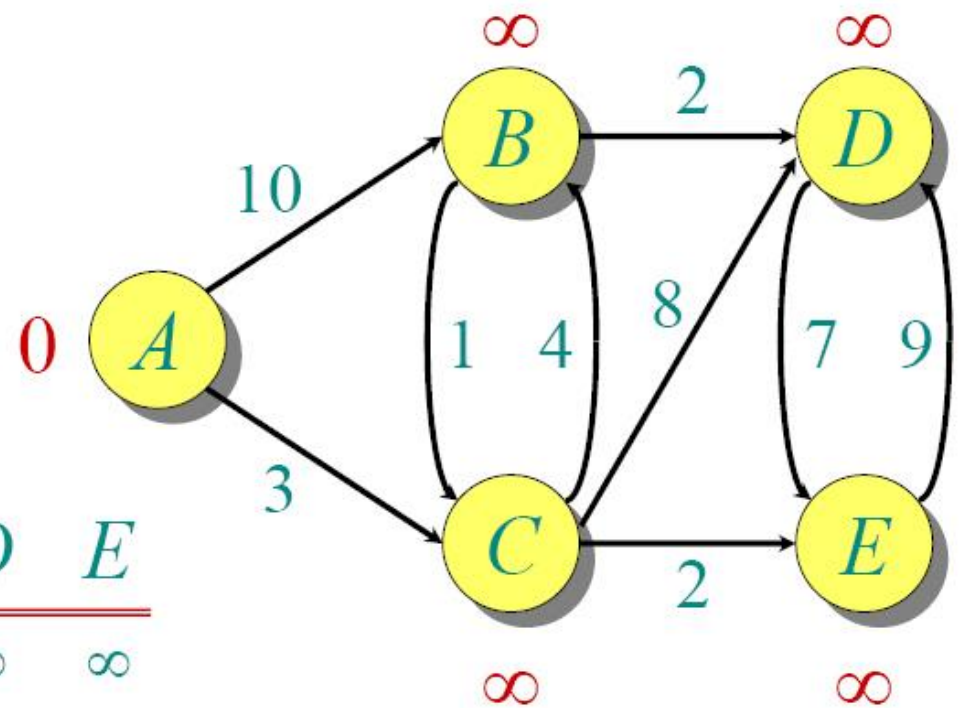
- The closest node to *A*, say *X*, must be adjacent to *A*
  - The next closest node, say *Y*, must be either adjacent to *A* or *X*
  - The third closest node to A must be either adjacent to *A* or *X* or *Y*, and so on

Dijkstra's algorithm - Pseudocode

```
dist[s] ← 0                          (distance to source vertex is zero)
for  all v ∈ V–{s}
      do  dist[v] ← ∞                (set all other distances to infinity)
S←∅                                  (S, the set of visited vertices is initially empty)
Q←V                                  (Q, the queue initially contains all vertices)
while Q ≠∅                           (while the queue is not empty)
do   u ← mindistance(Q, dist)        (select the element of Q with the min. distance)
    S ← S ∪ {u}                      (add u to list of visited vertices)
    for all v ∈ neighbors[u]         (and v should not b de-queued vertex)
          do  if   dist[u] + w(u, v) < dist[v]        (if new shortest path found)
                then   dist[v] ← dist[u] + w(u, v)    (set new value of shortest path)
                       p[v] = u
return dist
```

Dijkstra Animated Example

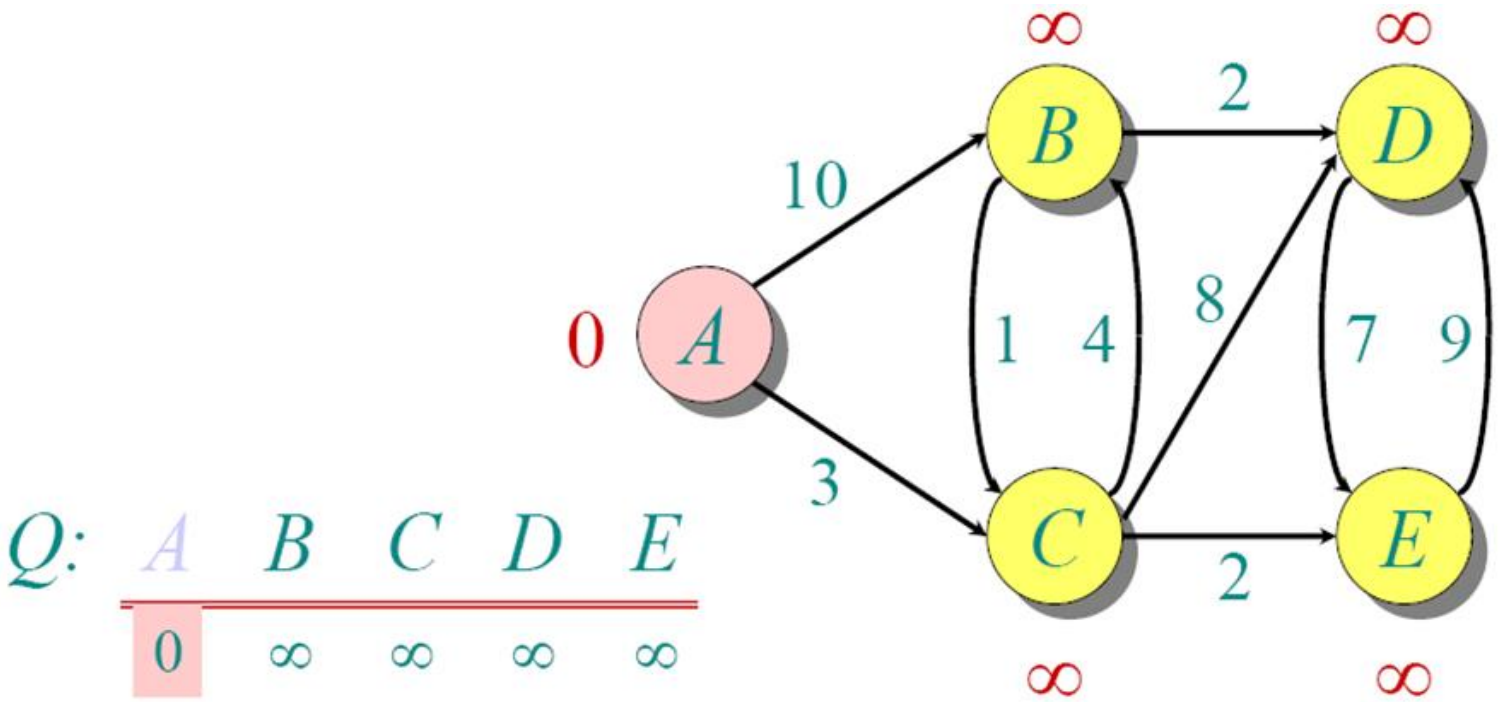
Initialize:



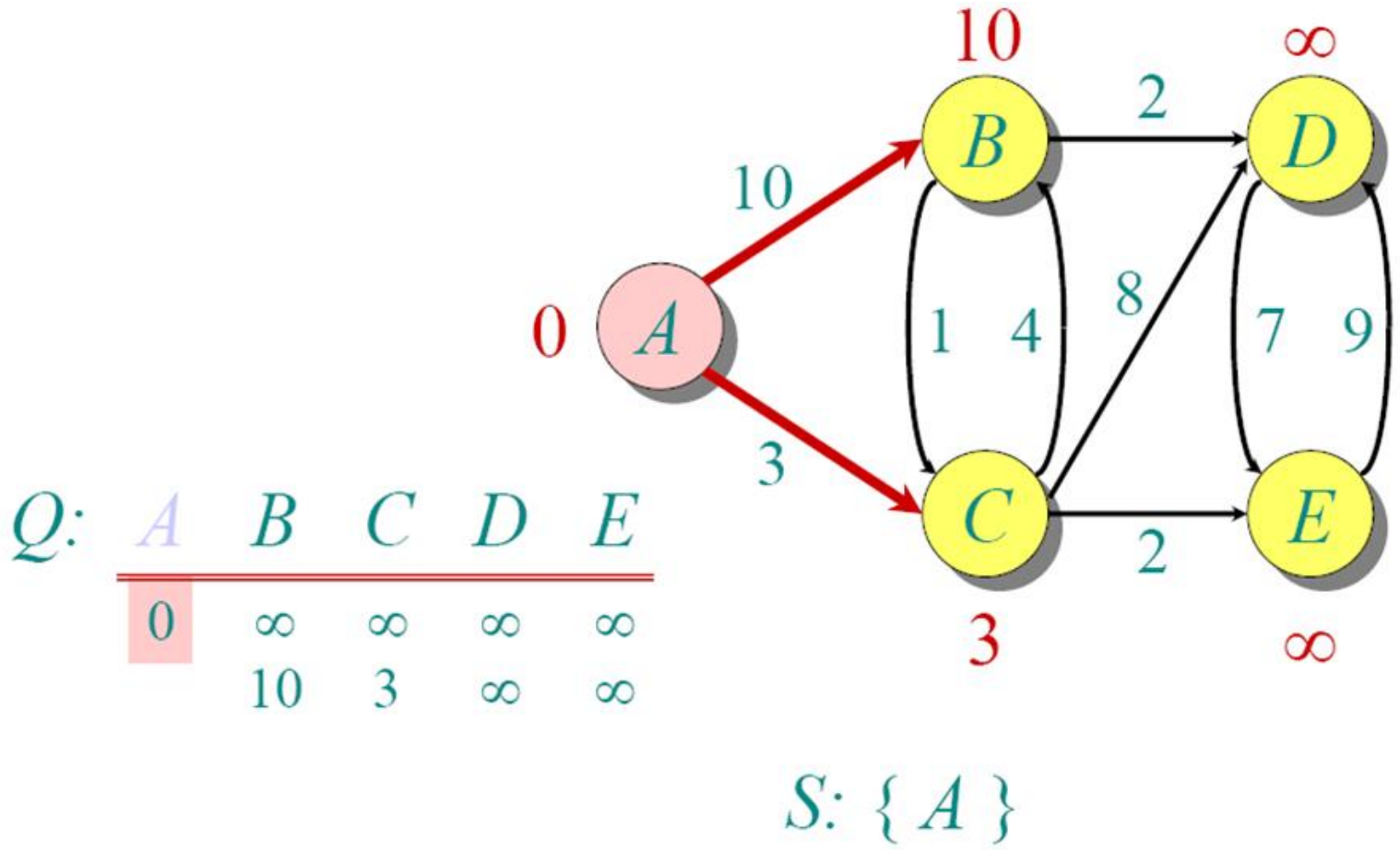
Q:	A	B	C	D	E
	0	∞	∞	∞	∞

S: {}

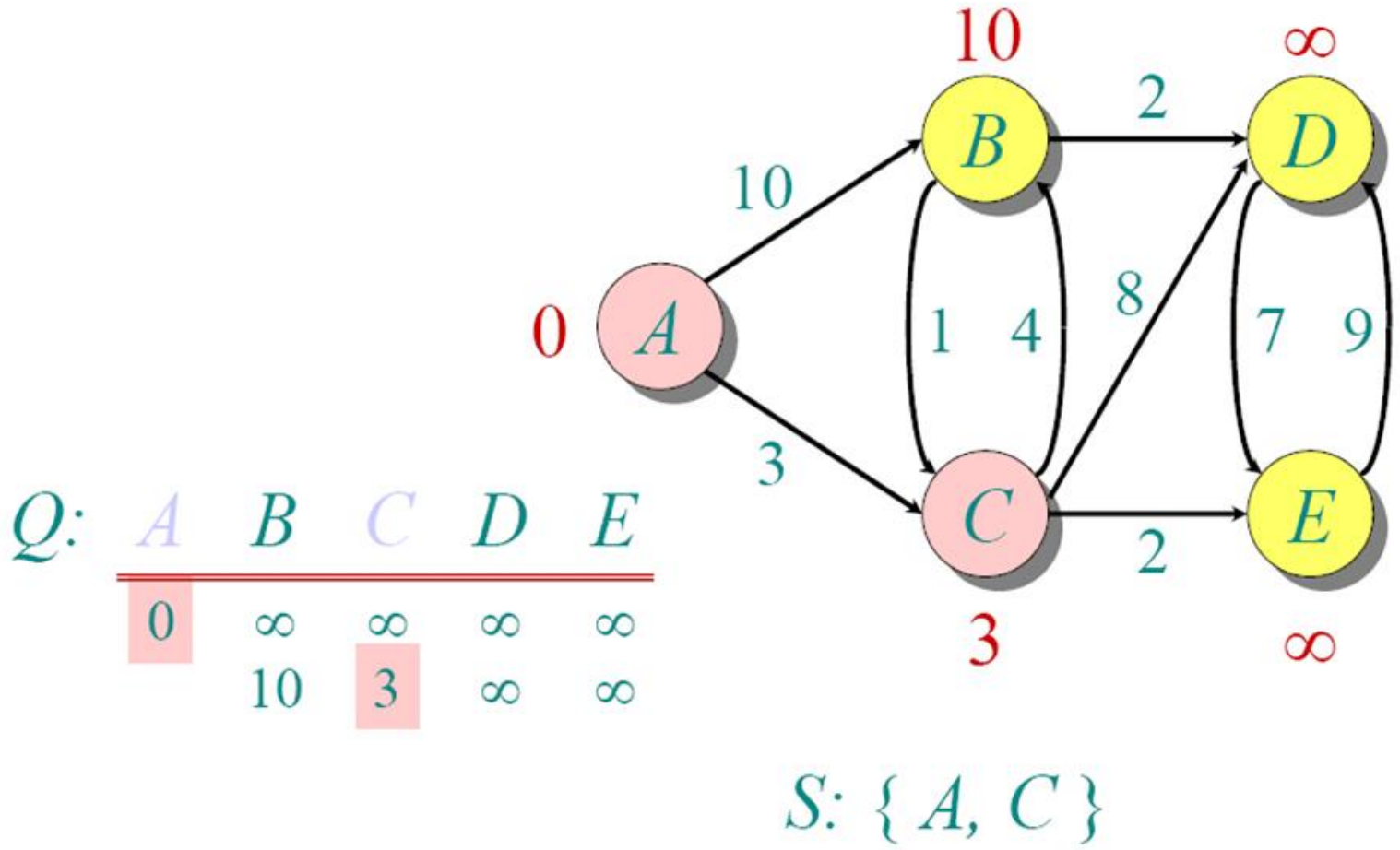
Dijkstra Animated Example



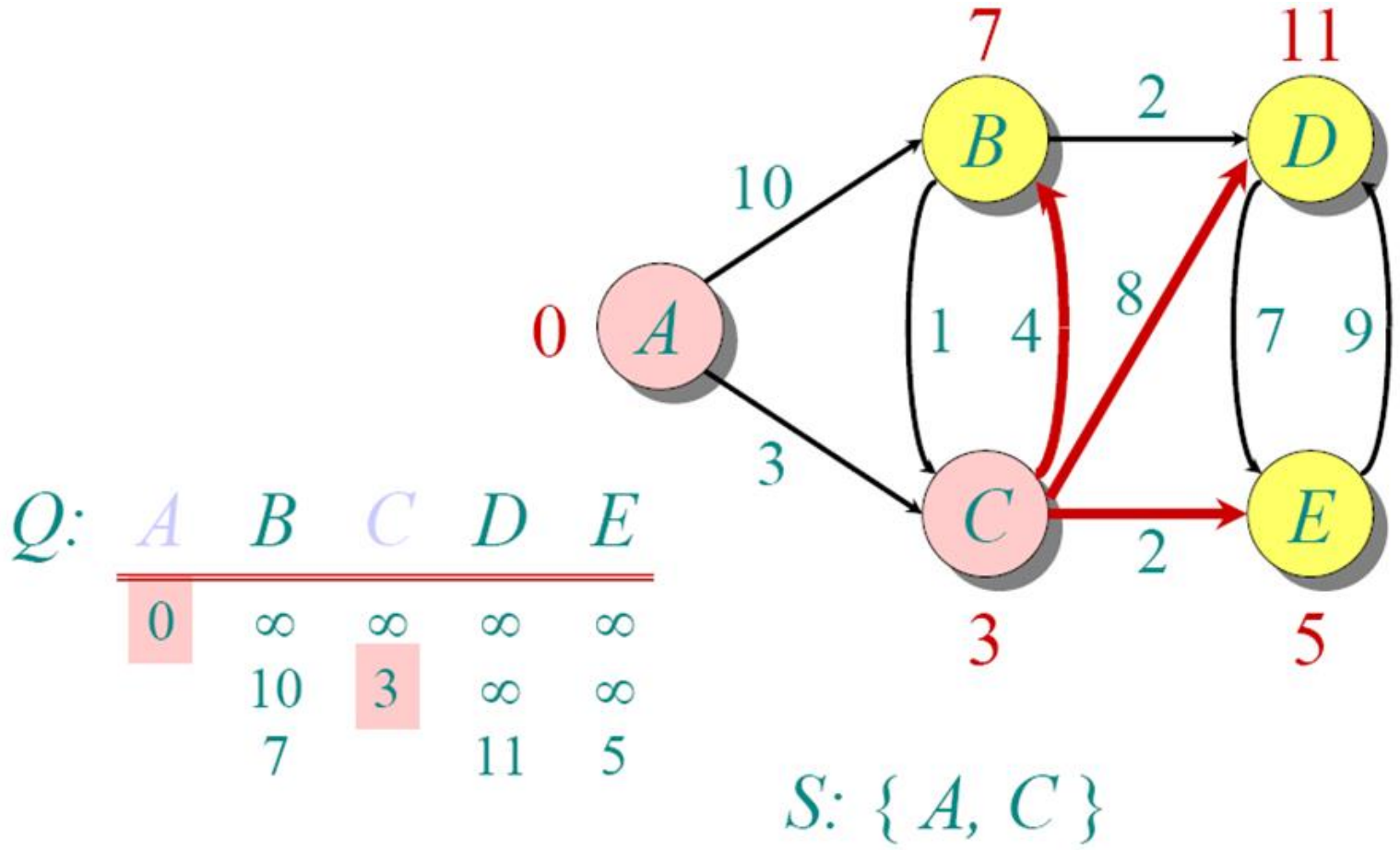
Dijkstra Animated Example



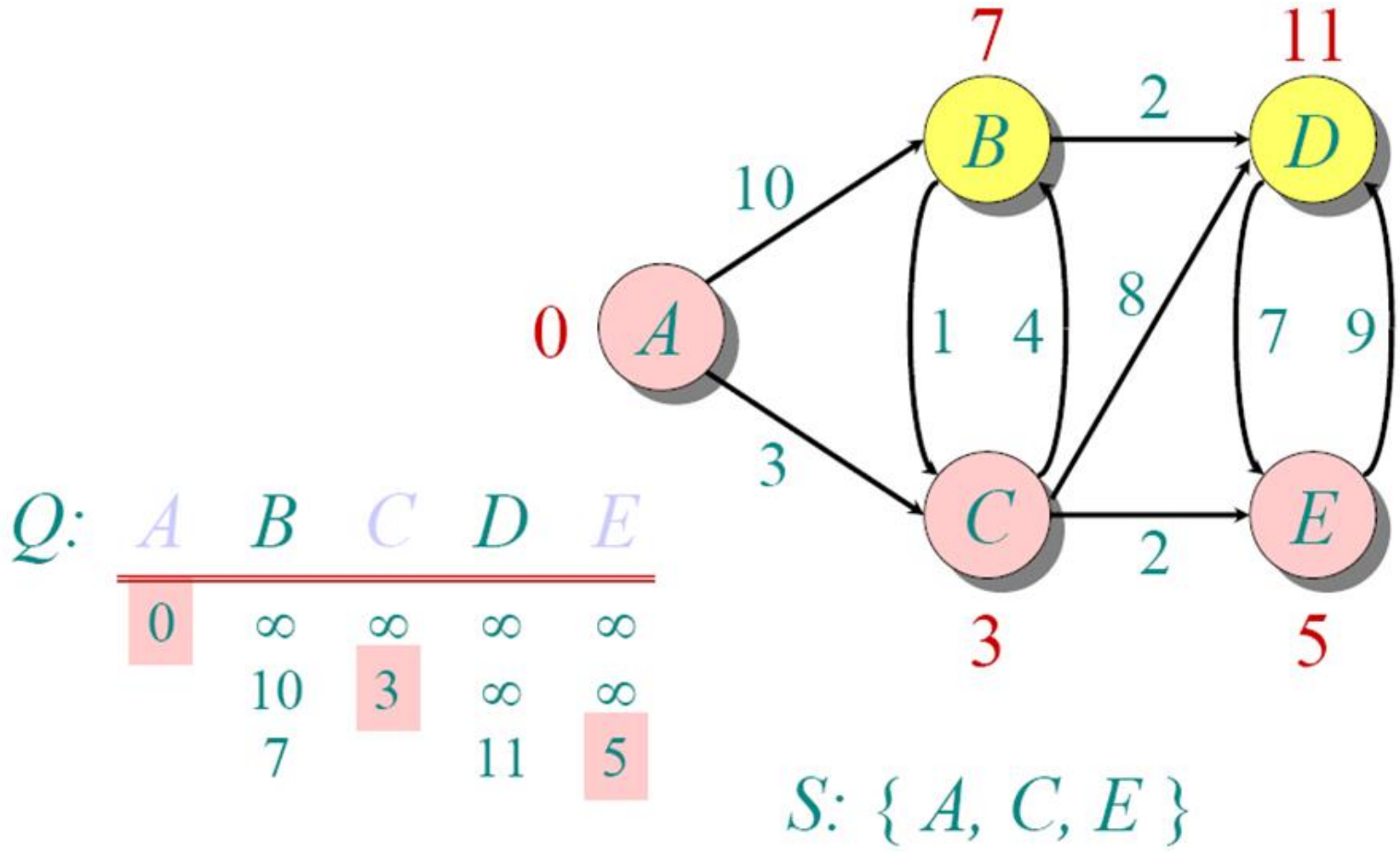
Dijkstra Animated Example



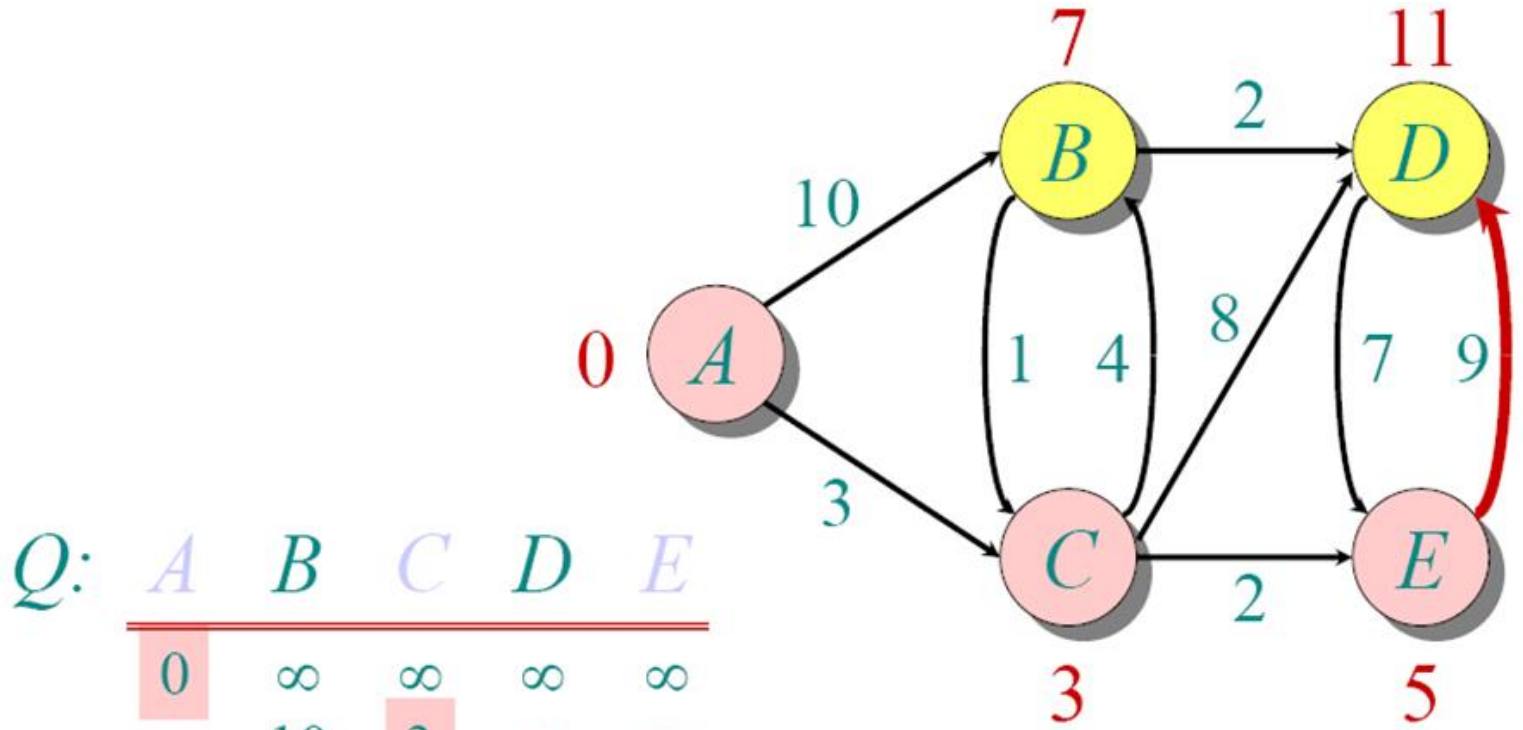
Dijkstra Animated Example



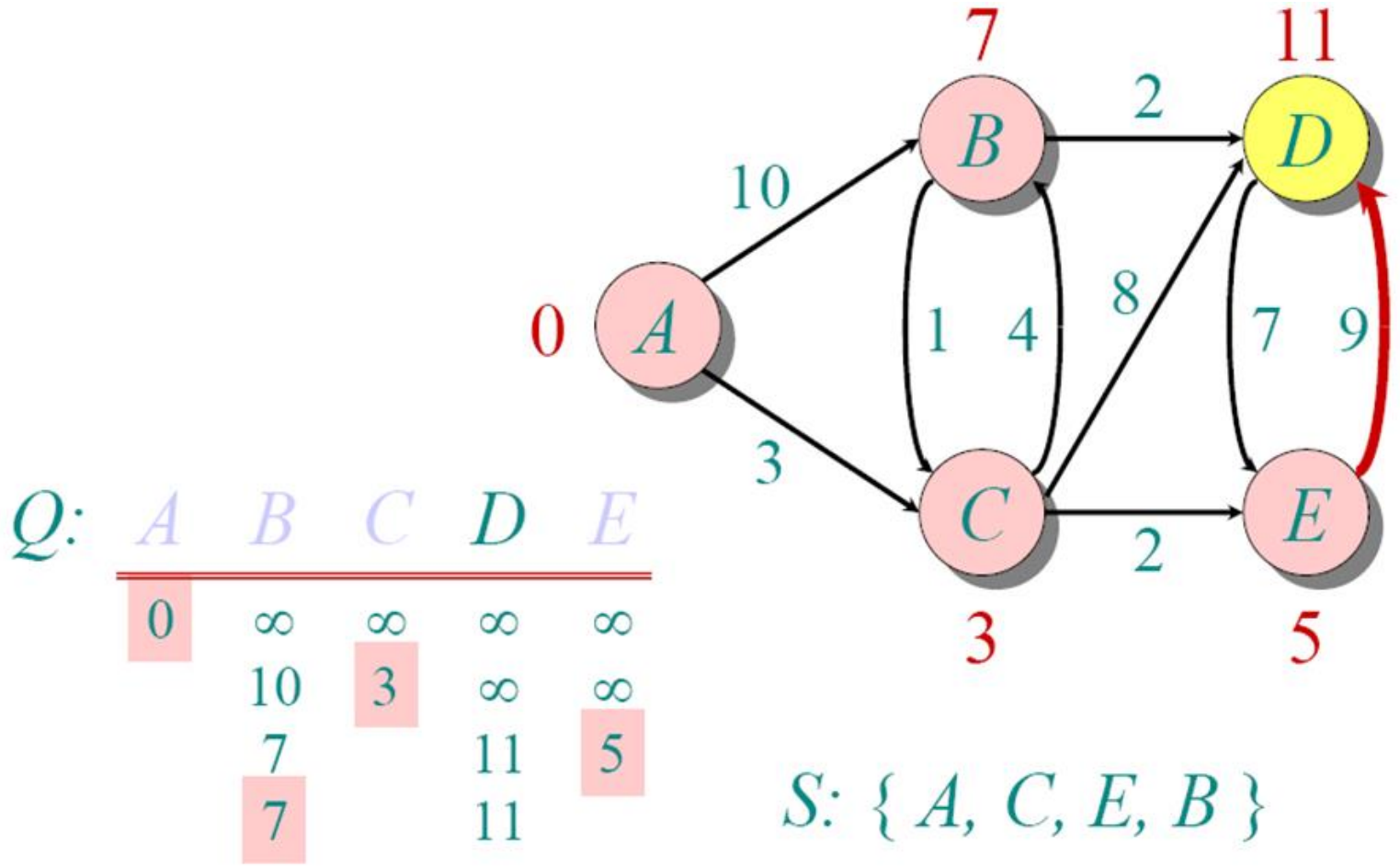
Dijkstra Animated Example



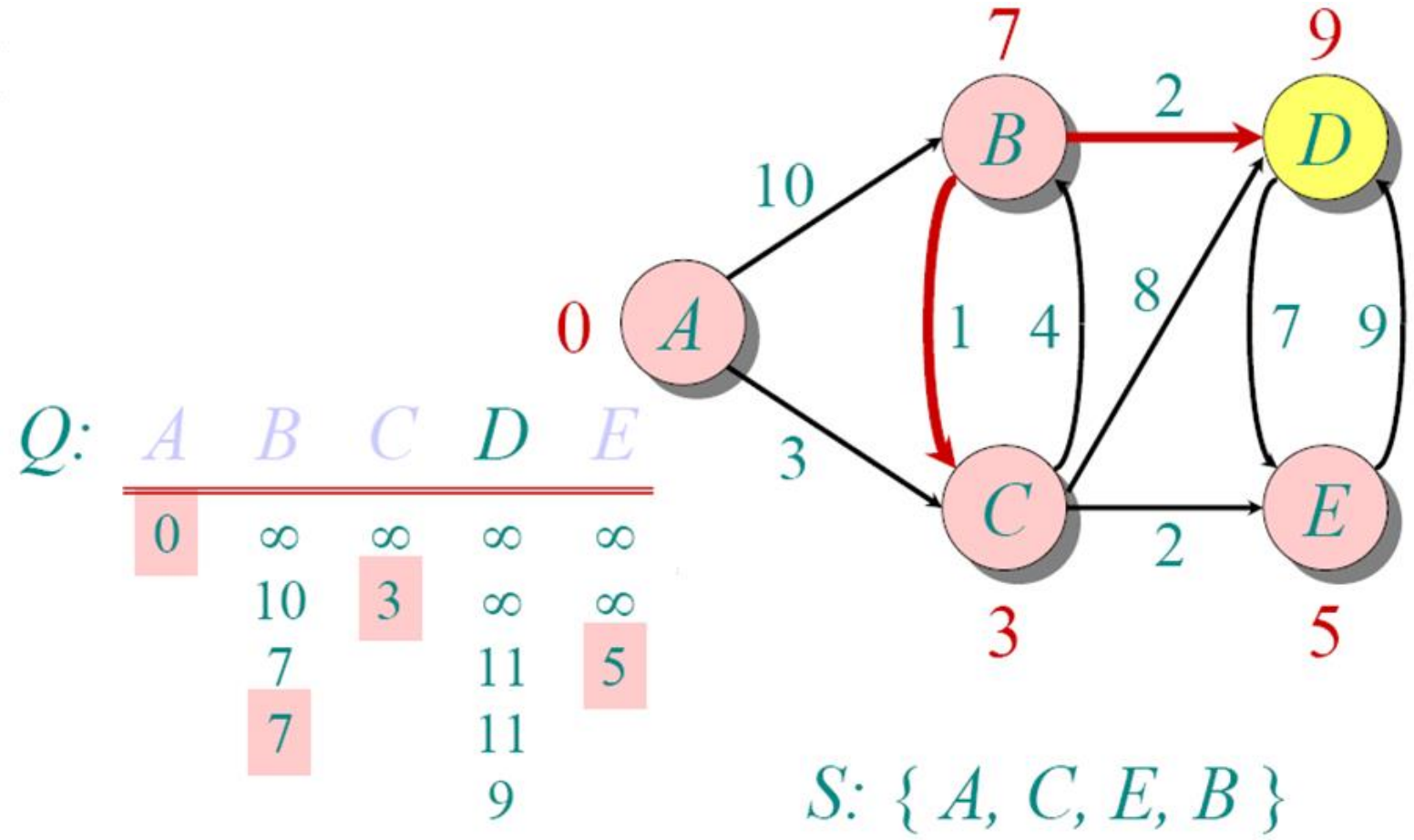
Dijkstra Animated Example



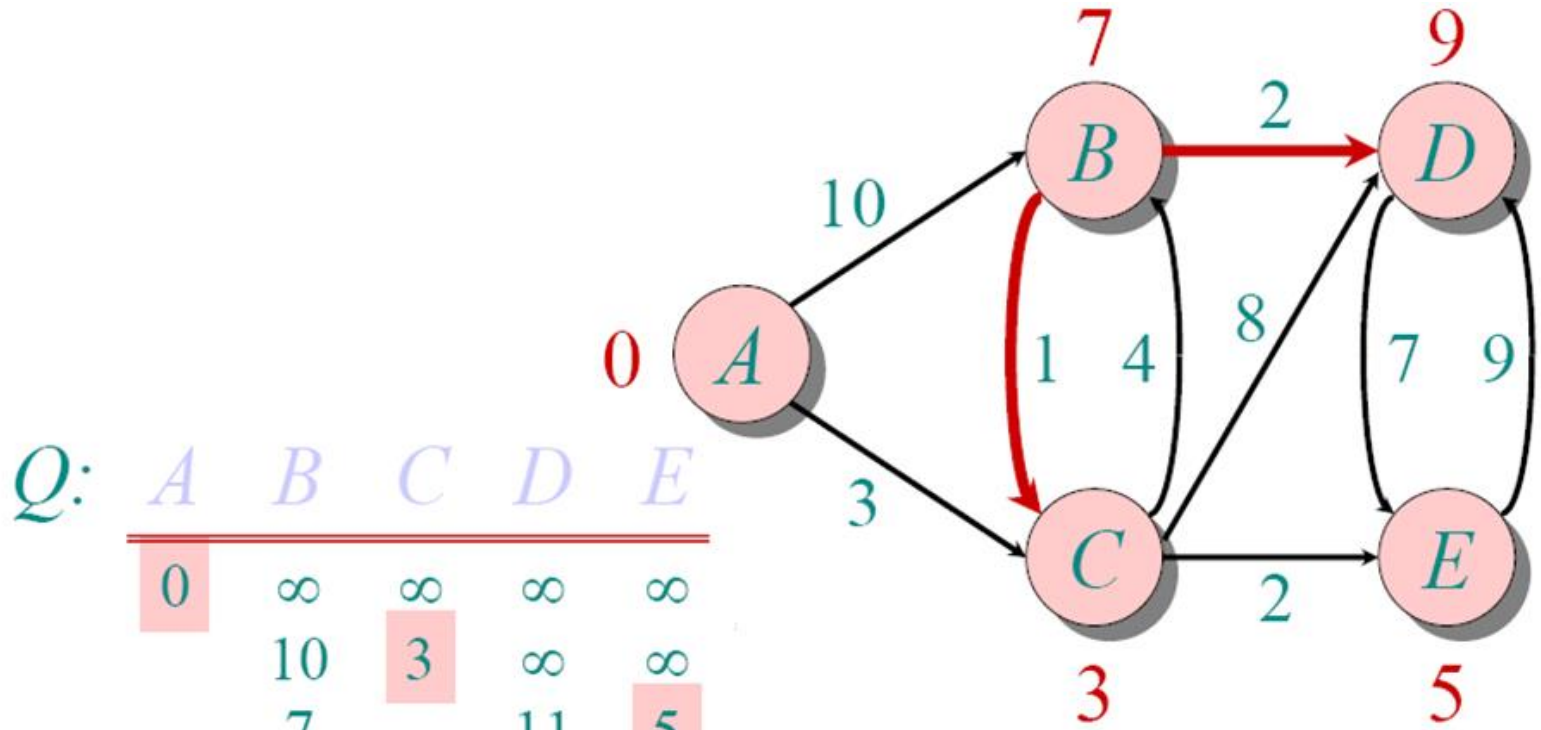
Dijkstra Animated Example



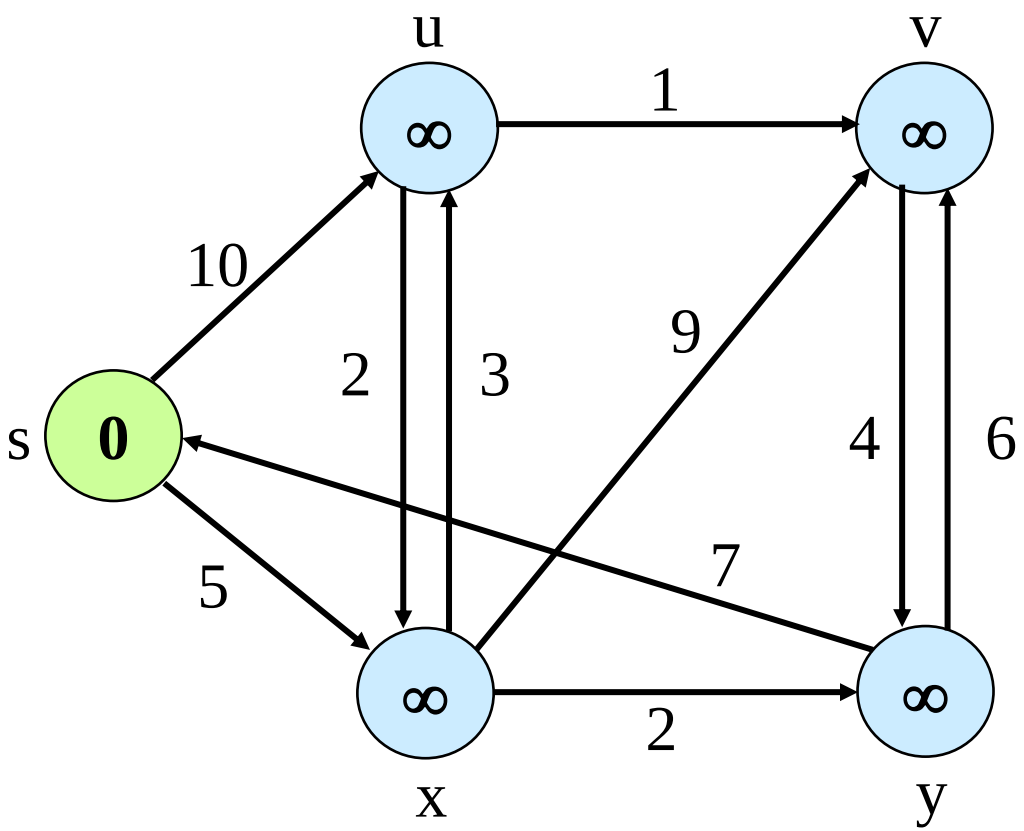
Dijkstra Animated Example



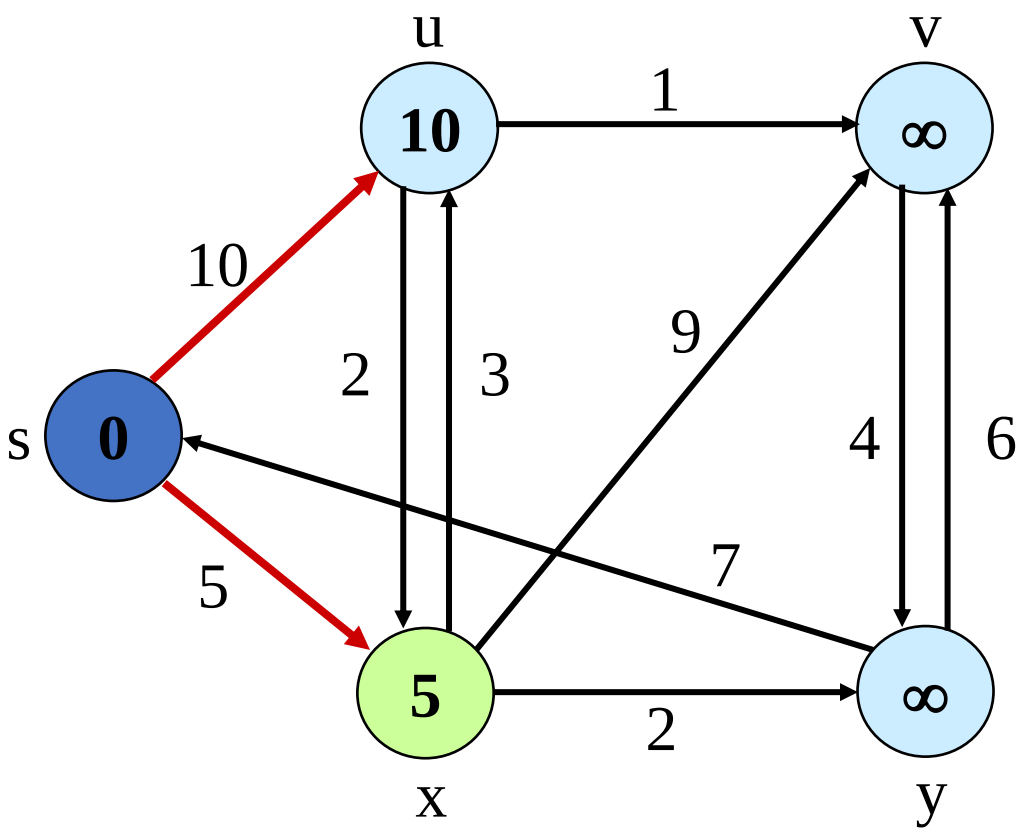
Dijkstra Animated Example



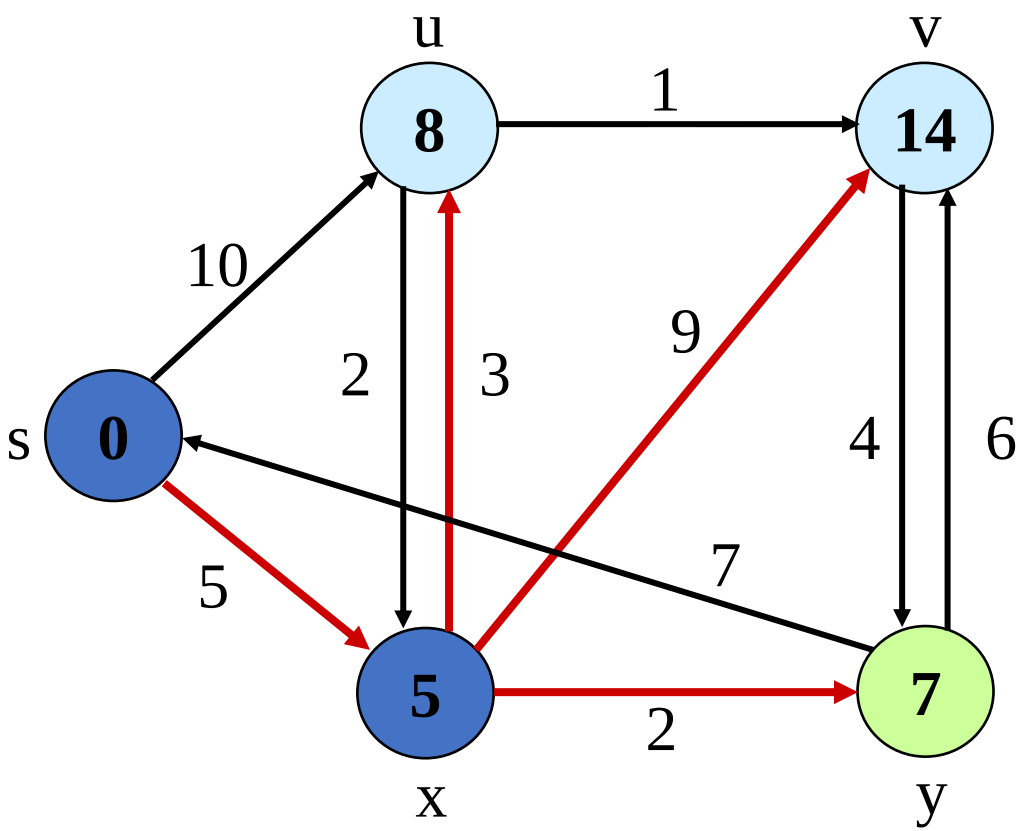
Class Task



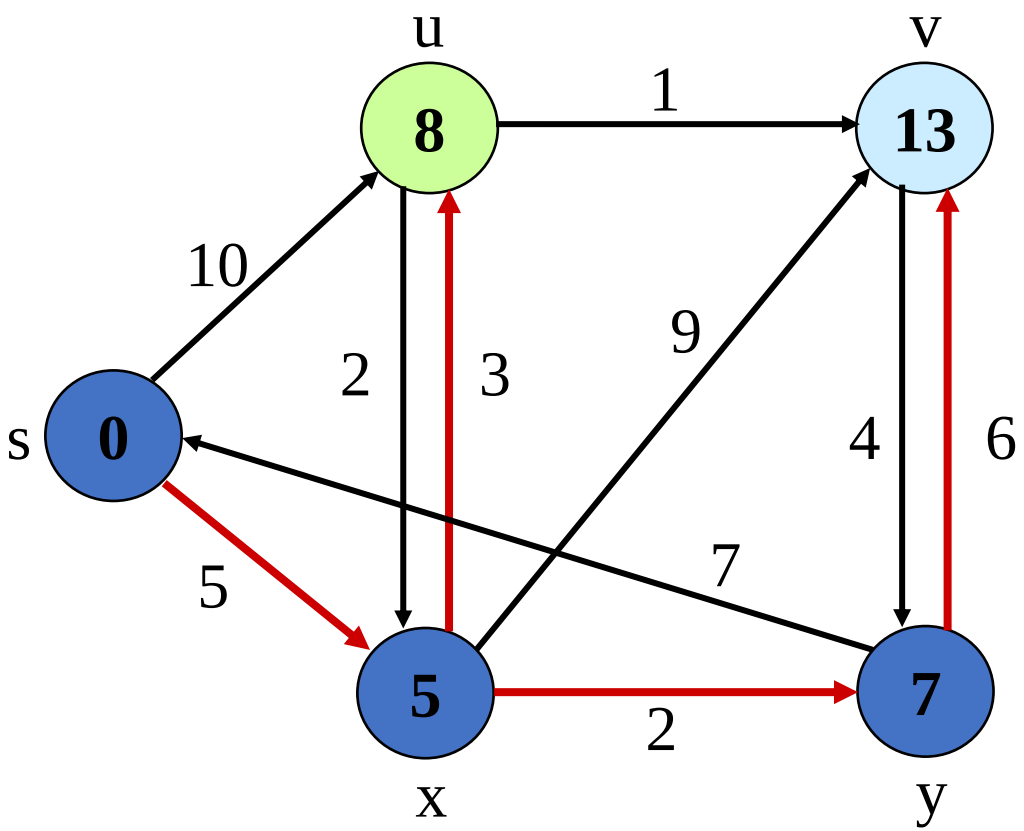
Example



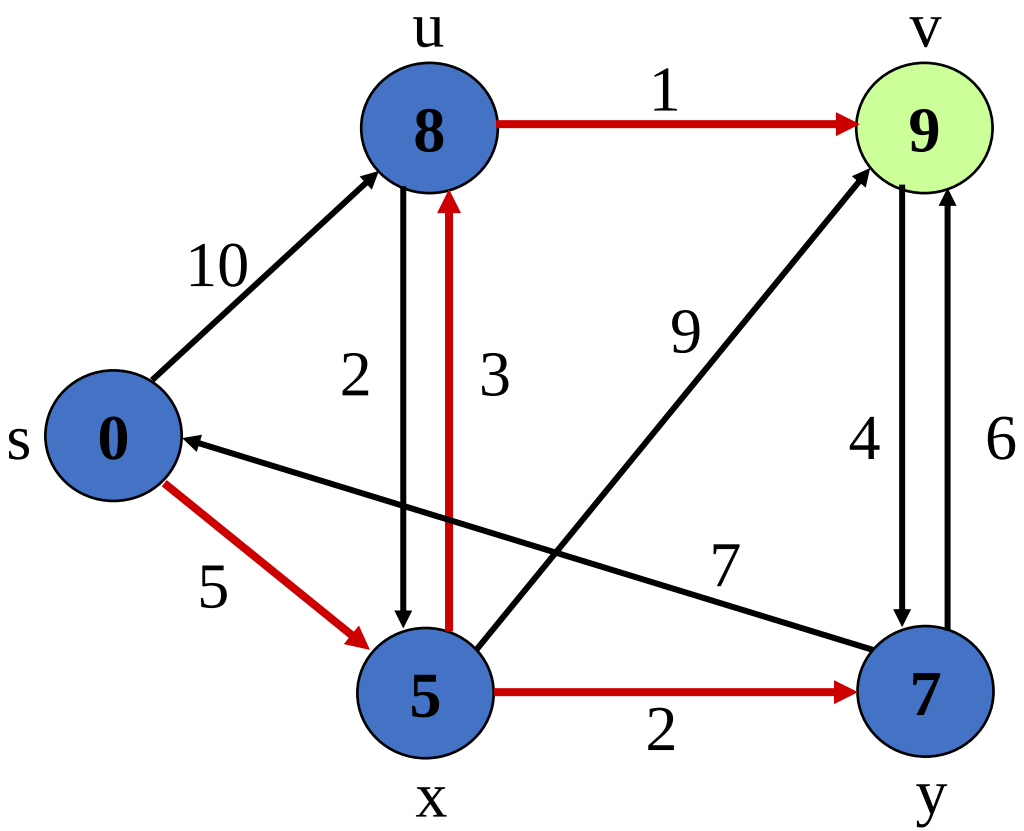
Example



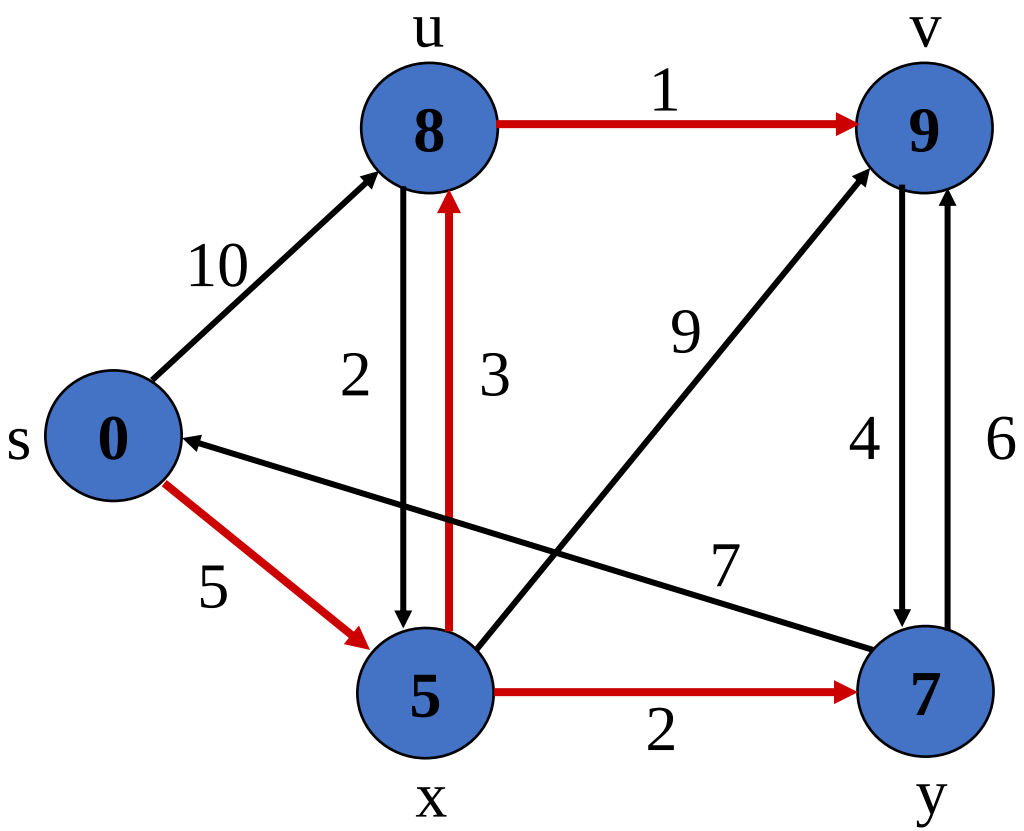
Example



Example



Example



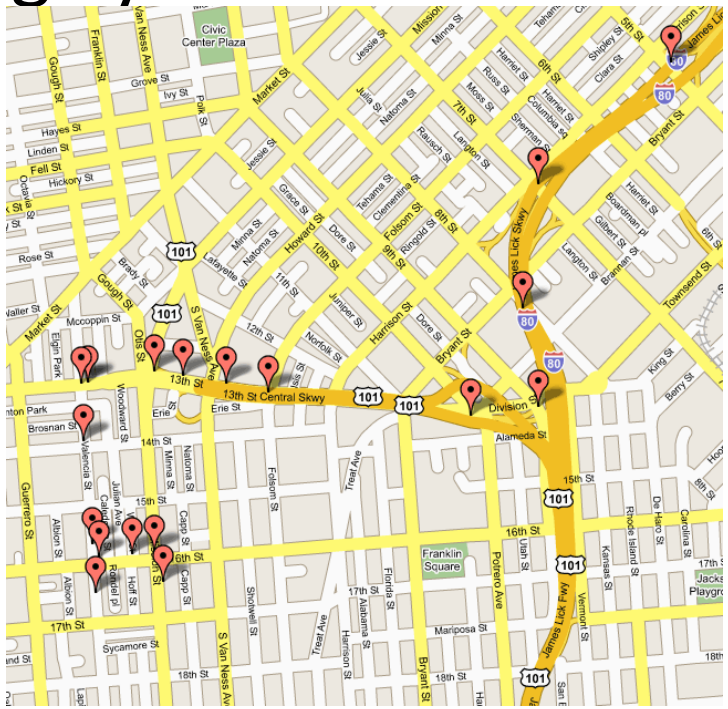
Dijkstra's Algorithm - Why use it?

- As mentioned, Dijkstra’s algorithm calculates the shortest path to every vertex.
- Therefore, anytime we want to know the optimal path to some other vertex from a determined origin, we can use Dijkstra’s algorithm.

- O(ElogV)

Applications of Dijkstra's Algorithm

- Mapping (Google Maps)
- Routing Systems



From Computer Desktop Encyclopedia
© 1998 The Computer Language Co. Inc.

Router A
Routing Table

To go to network:	Route via port #:
10.0.0.0	1
20.0.0.0	2
30.0.0.0	3
40.0.0.0	1

